

Analiza amortyzowana

Problem ogólny:

Na strukturze danych wykonywany jest ciąg operacji. Pytanie: ile wynosi koszt całego ciągu?

Równoważnie:

Ile wynosi pesymistyczny koszt pojedynczej operacji, jako średnia (arytmetyczna) kosztu całego ciągu.

Czyli: ile wynosi

koszt amortyzowany (pojedynczej) operacji.

Metody analizy amortyzowanej:

Metoda 1: zespolona (aggregate method)

Obliczamy koszt całego ciągu, dzielimy przez liczbę operacji.

Najczęściej:

- podczas liczenia kosztu całego ciągu nie analizujemy wprost kosztu pojedynczej operacji,
- zamiast tego szacujemy koszt wszystkich działań na strukturze danych podczas wykonywania całego ciągu operacji,
- a na koniec wykazujemy, że koszt całego ciągu operacji jest nie większy niż oszacowany koszt działań na strukturze danych (czyli: koszt działań dominuje w ciągu operacji, w sensie złożoności).

Przykład 1: multi-stos

stos z operacją wielokrotnego ściągania:

- $\text{push}(S, x)$ – wstaw na stos
- $\text{pop}(S)$ – usuń i zwróć element
- $\text{multipop}(S, k)$ – usuń k górnych elementów (wszystkie, jeśli $k \geq |S|$).

Koszt pojedynczej operacji:

- $\text{push}()$ i $\text{pop}()$ – $O(1)$,
- $\text{multipop}(k)$ – $O(k)$.

$\Rightarrow$ koszt ciągu n operacji na początkowo pustym stosie:

$$T(n) = O(nk) = O(n^2) \quad // \text{ "o duże", ale nie } \Theta!$$

Analiza dokładna:

szacujemy koszt całego ciągu operacji, przyglądając się strukturze danych, a nie pojedynczym operacjom:

n operacji, stos początkowo pusty $\Rightarrow$

$\leq n$ wstawień (bo WSZYSTKICH operacji jest $\leq n$) $\Rightarrow$

$\leq n$ usunięć elementów w całym ciągu (pop i multipop).

a ponieważ wstawienie 1 elementu jak i usunięcie 1 elementu kosztuje $O(1)$ $\Rightarrow$

koszt całego ciągu wynosi $O(n)$.

Wniosek:

koszt amortyzowany dowolnej operacji struktury "multi-stos" wynosi $O(n)/n = O(1)$.

Przykład 2: Licznik binarny.

$A[0..k-1]$ – tablica bitowa (licznik),
 $A[0]$ – bit najmniej znaczący

Operacja: $\text{Inc}(A)$ - zwiększ licznik o 1.

Problem:

Jaka jest złożoność ciągu n operacji $\text{Inc}(A)$ na liczniku początkowo pustym.

Początkowo:

wartość licznika = 0 (wszystkie bity wyzerowane)

$\text{Inc}(A)$:

```
i := 0;  
while i < k and A[i] = 1  
do  
    A[i] := 0;  
    i := i + 1;  
if i < k then A[i] := 1;  
end.
```

- pesymistyczny koszt pojedynczej operacji $\Theta(k)$
- z tego nie wynika że pesymistyczny koszt całego ciągu n operacji wynosi $\Theta(nk)$, lecz jedynie $O(nk)$.

Analiza dokładna

jako poprzednio – analizując zmiany w strukturze danych, a nie pojedyncze operacje:

Koszt całego ciągu = $\Theta(\text{liczba wszystkich zmian bitów})$.

bit nr 0 zmieniany przez każdą operację,

bit nr 1 zmienia się co 2. operację,

bit nr 2 zmienia co 4. operację,
bit nr 3 zmienia co 8. operację,
...
bit nr k zmienia co 2^k -tą operację.

Najwyższy numer bitu zmienianego wynosi $\lfloor \log n \rfloor$.

Koszt zmian bitów, i zarazem koszt całego ciągu operacji:

$$\begin{aligned} n/1 + n/2 + n/4 + \dots + n/2^{\log n} &< \\ &< n(1 + 1/2 + 1/4 + 1/8 + \dots) \\ &= 2n \end{aligned}$$

Zatem:

koszt amortyzowany operacji $\text{Inc}(A)$ wynosi
 $O(n)/n = O(1)$.

Metoda 2: księgowanie

- ustal (zgadnij) i przypisz każdej operacji pewien koszt - opłatę za wykonanie; każda operacja płaci tę kwotę;
- jeśli rzeczywisty koszt operacji jest mniejszy niż ustalony, to nadmiar kosztu magazynowany jest w elementach struktury danych ("księgowany");
- jeśli rzeczywisty koszt operacji jest większy niż ustalony, to brakującą różnicę pokrywa się pobierając potrzebną ilość z magazynu;
- analiza jest poprawna jeśli wykażemy, że zawsze wystarcza zgromadzonych środków na pokrycie rzeczywistego kosztu operacji;
- ustalony na początku koszt operacji to właśnie koszt amortyzowany.

Przykład: Multi-stos

- definiujemy koszt amortyzowany $\text{push}() = 2\text{zł.}$, tyle ta operacja 'płaci',
- z tych 2 zł. 1 zł pokrywa rzeczywisty koszt wykonania operacji,
- pozostałe 1zł księgujemy przy wstawionym na stos elemencie,
- zaksięgowanymi złotówkami elementy 'płacą' za ewentualne usunięcie w $\text{pop}()$ lub $\text{multipop}()$; pieniędzy tych zawsze wystarcza, bo:
 - w dowolnym momencie do każdego elementu na stosie przypisany jest kredyt 1 zł na ewentualne późniejsze usunięcie tego elementu ze stosu;
- zatem koszt amortyzowany $\text{pop}()$ i $\text{multipop}()$ w takim ciągu (przy tak przeprowadzonej analizie) jest zerowy (obie operacje nic nie muszą płacić).

Wniosek - właściwy wynik analizy amortyzowanej:

koszt całego ciągu n operacji na początkowo pustym multi-stosie wynosi $O(n)$ – bo:

- od każdej operacji pobrano nie więcej niż 2 zł, czyli w sumie od wszystkich operacji pobrano nie więcej niż $2n$ zł,
- pobrane opłaty wystarczyły na realizację całego ciągu.

Licznik binarny:

- koszt amortyzowany $\text{Inc}()$ kładziemy równy 2zł.,
- wykonując operację:
 - za ustawienie jednego bitu na 1 płacimy 1zł. pochodzące z opłaty wniesionej przez operację,
 - pozostałe 1 zł. z tej opłaty księgujemy do ustawionego właśnie bitu,
 - za zerowanie najmniej znaczących bitów równych 1 płacimy złotówkami zaksięgowanymi wcześniej do tych bitów.

Zatem w dowolnym momencie do każdego ustawionego bitu przypisany jest kredyt 1 zł. na ewentualne późniejsze zerowanie tego bitu.

Wniosek:

koszt ciągu n operacji $\text{Inc}()$ na początkowo wyzerowanym liczniku wynosi $O(n)$.

Obserwacja:

- w metodzie zespolonej
 - szacujemy najpierw koszt całego ciągu,
 - a potem wyliczamy koszt amortyzowany operacji, jako zwykłą średnią arytmetyczną;
- w metodzie księgowania
 - odgadujemy koszt amortyzowany (pojedynczej) operacji,
 - dowodzimy, że jest poprawny ("w dowolnym momencie wystarcza środków na wykonanie kolejnej operacji"),
 - a na końcu wyliczamy koszt pesymistyczny ciągu, jako nie większy niż koszt amortyzowany pojedynczej operacji pomnożony przez liczbę operacji.

Metoda potencjału

Definiujemy ("zgadujemy") funkcję opisującą sumaryczny kredyt zgromadzony w strukturze (intuicyjne odwołanie do metody księgowania). Z niej wyliczamy koszt amortyzowany operacji.

Oznaczenia:

- D_i – struktura danych po i -tej operacji,
- $\Phi(D_i) = \Phi_i$ – potencjał (kredyt) struktury po i -tej operacji,
- zakładamy, że $\Phi_0=0$ oraz $\Phi_i \geq 0, \forall i$,
- c_i – koszt rzeczywisty i -tej operacji

Definiujemy koszt amortyzowany i -tej operacji jako:

$$(*) \quad \hat{c}_i = c_i + \Delta(\Phi) = c_i + \Phi_i - \Phi_{i-1} ,$$

czyli suma kosztu rzeczywistego i przyrostu potencjału w wyniku tej operacji – odpowiada to idei metody księgowania.

Sumując (*) po wszystkich operacjach otrzymujemy:

$$\hat{c}_1 + \dots + \hat{c}_n = c_1 + \dots + c_n + \Phi_n - \Phi_0$$

(pozostałe składniki Φ_i się zredukowały - suma teleskopowa),

a ponieważ $\Phi_n - \Phi_0 \geq 0$ otrzymujemy:

Lemat (o poprawności metody amortyzowanej)

Dla dowolnej funkcji potencjału zdefiniowanej jak wyżej koszt amortyzowany ciągu operacji ogranicza od góry koszt rzeczywisty tego ciągu operacji.

Jeśli więc po odpowiednim zdefiniowaniu funkcji potencjału obliczymy, że

- koszt amortyzowany operacji i -tej wynosi $O(b_i)$ (b_i nie musi być stałą)

to wynika stąd, że

- koszt rzeczywisty całego ciągu operacji wynosi $O(\sum_{i=1\dots,n} b_i)$.

Przykład: **multi-stos**.

Φ_i := liczba elementów na stosie po operacji i -tej.

Operacja push():

- przed operacją: $\Phi_{i-1} = p$,
- po operacji: $\Phi_i = p+1$,
- $c_i = 1$,

$$\text{zatem } \hat{c}_i = 1 + (p+1) - p = 2.$$

Operacja pop():

- $\Phi_{i-1} = p$,
- $\Phi_i = p-1$,
- $c_i = 1$,

$$\text{zatem } \hat{c}_i = 1 + (p-1) - p = 0.$$

Formalnie przyjmujemy: $\hat{c}_i(\text{pop}()) = O(1)$, czyli stały.

Operacja multipop(k):

$$\Phi_{i-1} = p, \Phi_i = p-k, c_i = k, \text{ zatem}$$

$$\hat{c}_i = k + (p-k) - p = 0$$

Multipop() też kosztuje 0 - formalnie $O(1)$.

Ćwiczenie: wykonaj analizę licznika binarnego metodą potencjału. Wskazówka: Φ_i = liczba bitów równych 1.

Tablice dynamiczne

najprostsza, typowa wersja problemu:

- wykonujemy ciąg n operacji "wstaw nowy element do tablicy" ($\text{insert}(x)$);
- n nie jest znane, więc nie znamy z góry wielkości potrzebnej tablicy;
- tablica początkowo jest pusta, po wstawieniu pierwszego elementu ma wielkość 1;
- za każdym razem gdy tablica się przepełnia, alokujemy nową tablicę wielkości $s_{i+1} = 2 \cdot s_i$ i przepisujemy wszystkie dotychczas zgromadzone elementy do nowej tablicy;
- koszt każdego podstawienia $A[k] := x$ wynosi 1,
- koszt pozostałych operacji wynosi zero.

Zauważmy:

koszt pesymistyczny pojedynczej operacji wynosi $\Theta(n)$, bo jeśli n -ta operacja wymaga alokacji nowej tablicy to $n-1$ elementów należy przepisać do nowej tablicy.

Analiza **metodą zespoloną**:

- koszt całego ciągu jest równy liczbie wykonań wszystkich instrukcji postaci $A[k] := x$;
- przypadek pesymistyczny (łatwo to zauważyć):
 $n-1$ jest potęgą dwójki;
- operacje wstawienia nowego elementu kosztowały n ;
- alokowano nowe tablice o rozmiarach kolejno
 $1, 2, 4, 8, \dots, n-1, 2(n-1)$;
- liczba (starych) elementów przepisywanych po kolejnych alokacjach wynosiła kolejno
 $0, 1, 2, 4, 8, \dots, n-1$;
- co kosztowało w sumie
 $1+2+4+8+\dots+n-1 = 2(n-1)-1 = 2n-3$;

- cały ciąg kosztuje więc co najwyżej

$$n + (2n-3) = 3n-3;$$
- a więc jedna operacja średnio kosztuje nie więcej niż

$$(3n-3)/n \leq 3 = O(1).$$

Analiza **metodą potencjału**:

Definiujemy:

- s_i – wielkość tablicy po i operacjach wstawienia,
- n_i – liczba elementów w tablicy,
 jeśli zaczynamy od pustej tablicy to $n_i = i$;

Kładziemy $\Phi_i = 2(n_i - s_i/2)$
 $= 2n_i - s_i$,

Wyjaśnienie:

- zamierzamy dowieść, że wystarczy obciążyć każdą operację wstawienia stałym kosztem;
- część tego kosztu zużywać na wstawienie w sytuacji gdy nie ma relokacji;
- pozostałą akumulować i pokrywać z niej koszt późniejszego przepisywania;
- bezpośrednio po przepisywaniu potencjał (kredyt) może spaść do zera (bo pokryto nim przepisywanie);
- kolejne dopisywane elementy muszą powodować odkładanie kredytu na przyszłe przepisywanie siebie i tych elementów, które już były wcześniej w tablicy i ich kredyt się skończył;
- zatem kredyt odkładany po wstawieniu każdego elementu musi być równy 2;
- zaś wielkość $n_i - s_i/2$ to liczba elementów wstawionych po ostatniej relokacji.

Obliczamy $\hat{c}_i$ - koszt amortyzowany i-tej operacji wstawiania przy tak zdefiniowanym potencjale.

Z definicji:

$$\hat{c}_i = c_i + \Phi_i - \Phi_{i-1}.$$

Oczywiście: $n_i = n_{i-1} + 1$

Przypadek 1:

- relokacja nie była potrzebna

$$s_i = s_{i-1}, c_i = 1$$

$$\begin{aligned}\hat{c}_i &= c_i + (2n_i - s_i) - (2n_{i-1} - s_{i-1}) \\ &= 1 + (2(n_{i-1} + 1) - s_{i-1}) - (2n_{i-1} - s_{i-1}) \\ &= 3\end{aligned}$$

Przypadek 2:

- wykonywana jest relokacja

$$n_{i-1} = s_{i-1}, s_i = 2s_{i-1}, c_i = n_i.$$

$$\begin{aligned}\hat{c}_i &= c_i + (2n_i - s_i) - (2n_{i-1} - s_{i-1}) \\ &= n_i + (2n_i - s_i) - (2n_{i-1} - s_{i-1}) \\ &= n_{i-1} + 1 + 2n_{i-1} + 2 - 2n_{i-1} - 2n_{i-1} + n_{i-1} \\ &= 3\end{aligned}$$

Zatem:

Koszt amortyzowany (pojedynczej) operacji wstawienia elementu do tablicy zwiększanej dynamicznie jest stały.

Tablice dynamiczne z operacją wstawiania i usuwania

Operacja wstawiania - jak powyżej:

- czekamy z relokacją aż tablica wypełni się całkowicie;
- alokując nową tablicę przepisujemy wszystkie elementy i kasujemy starą tablicę.

Realizacja operacji usuwania:

- jeśli zajętość tablicy spada poniżej $1/4$, to przydzielamy nową tablicę wielkości połowy dotychczasowej i przepisujemy do niej wszystko,
- zakładamy, że koszt usunięcia elementu (gdy nie ma przepisywania do nowej mniejszej tablicy) wynosi 1.

Fakt: Koszt amortyzowany pojedynczej operacji tablicy dynamicznej (wstawienia, usunięcia) wynosi $O(1)$.

Dowód metodą potencjału (ćwiczenie).

Wskazówka: funkcja potencjału dla przypadku zajętości tablicy mniejszej niż $1/4$ powinna mieć inną postać niż dla zajętości większej niż $1/2$.